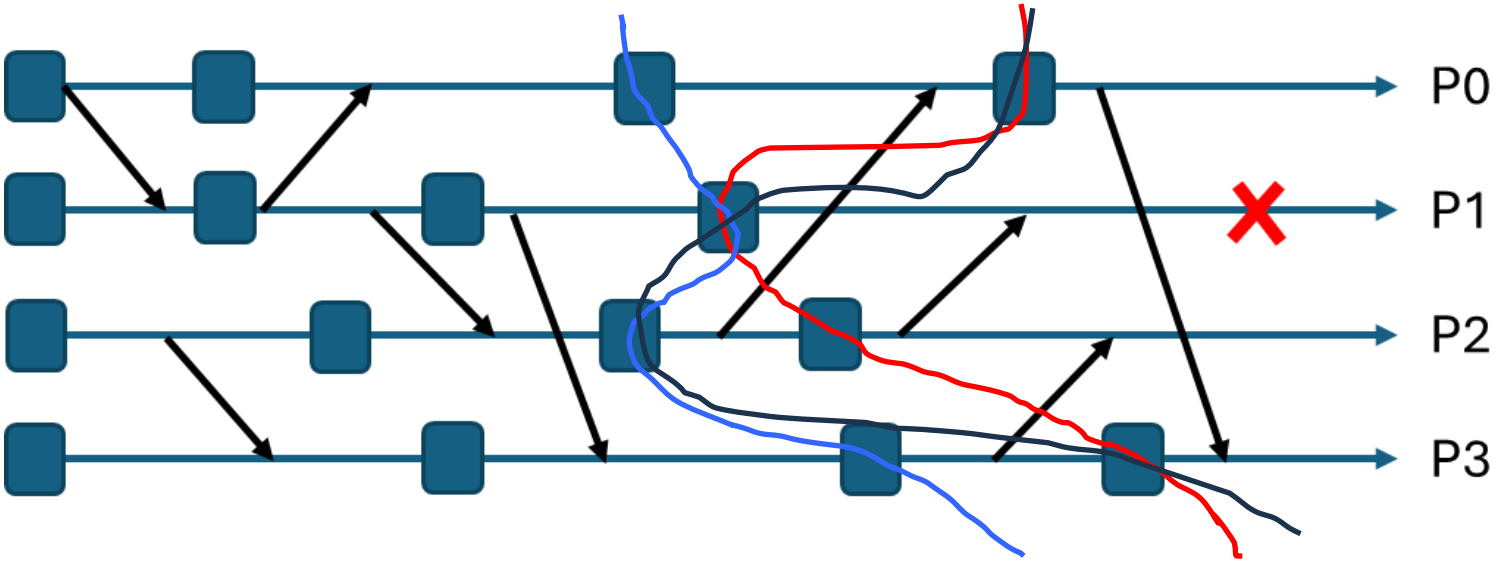
Home Exercise: Checkpointing

- Find the consistent cuts in the figure below (passing by )
- The recovery line that you found is consistent, strongly consistent or inconsistent?



Coding Theory
(Information Redundancy)

2110473 Fault Tolerant Computing

Code Design for Dependable Systems, Eiji Fujiwara, JOHN WILEY & SONS, ISBN-13 978-0-471-75618-7, 2006
Notes on Coding Theory, J.I.Hall, Michigan State University, 2010
and Several Sources

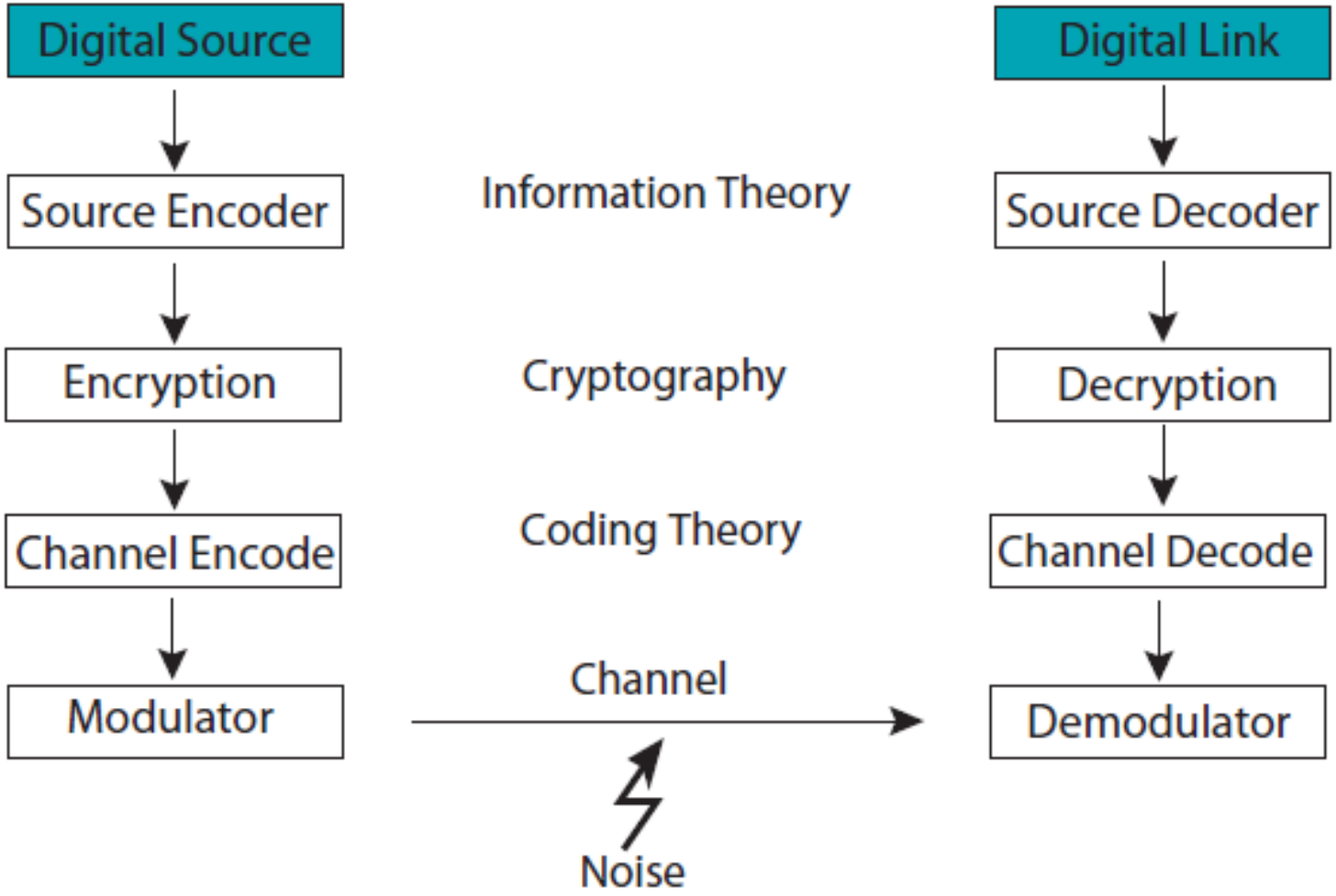
Contents

- Introduction
- Definition of Code
- Hamming Distance
- Linear/Cyclic Codes & Implementation
- Error Controls for Digital Systems
- Error Correcting Code
- Error Detecting Code: Checksum Code, Residue Code, Berger Code, Unordered Code

Introduction

- Coding Theory
  - The study of the properties of codes and their fitness for a specific application
    - Data Compression
    - Error Correction/ Error Control
    - Cryptography
    - Network Coding

Various Stages of Data Processing



Reliable Communication



- In practice, most media are not perfect, *Noisy channels*:
  - *Fiber-optic line, Satellite link, Hard disk, …*
- Can we recover the original message (without errors) from a noisy code string?
- We focus two things:
  - *Length of the code string*
  - *Probability of error*

Coding Theory

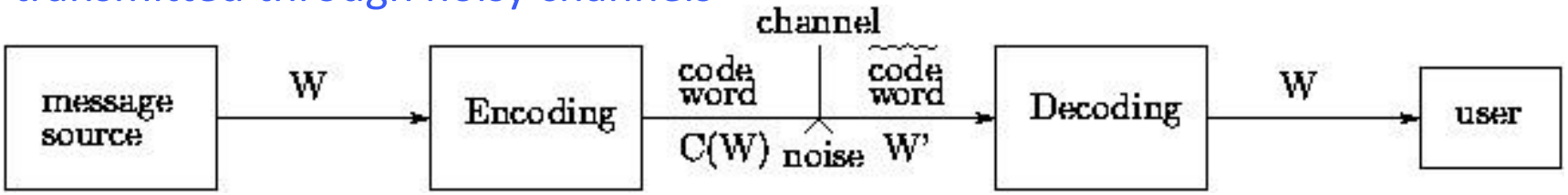
- Coding theory - theory of error correcting codes - is one of the most interesting and applied part of mathematics and informatics
- All real communication systems that work with digitally represented data, as CD players, TV, fax machines, internet, satellites, mobiles, require to use error correcting codes because all real channels are, to some extent, noisy - due to interference caused by environment
  - Coding theory problems are therefore among the very basic and most frequent problems of storage and transmission of information
  - Coding theory results allow to create reliable systems out of unreliable systems to store and/or to transmit information
  - Coding theory methods are often elegant applications of very basic concepts and methods of (abstract) algebra

Coding Theory/ Cryptography/ Steganography

- Coding Theory
  - Methods to protect information against a noise
  - Theory of *Error Correcting Codes (Error Control Codes)*
- Cryptography
  - Methods how to protect information against an enemy (or an unauthorized user)
  - Battles between codemakers and codebreakers
- Steganography
  - Methods to hide important information in innocently looking information (and that can be used to protect intellectual properties)

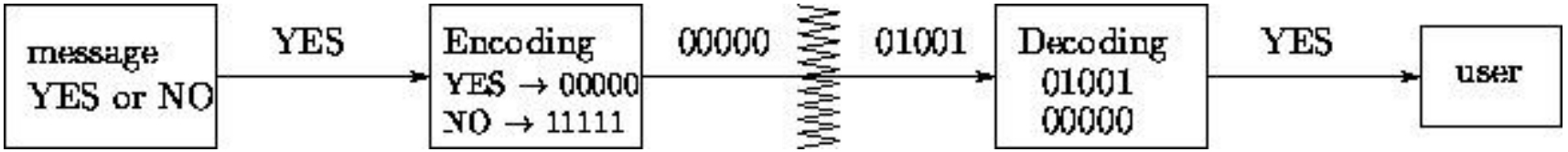
Basic Concept

Error-correcting codes are used to correct messages when they are transmitted through noisy channels



Error correcting framework

Example



A code C over an alphabet Σ is a subset of Σ^*($C \subseteq \Sigma^*$)

A q -nary code is a code over an alphabet of q -symbols

A binary code is a code over the alphabet {0,1}

Examples of codes C_1 = {00, 01, 10, 11} C_2 = {000, 010, 101, 100}

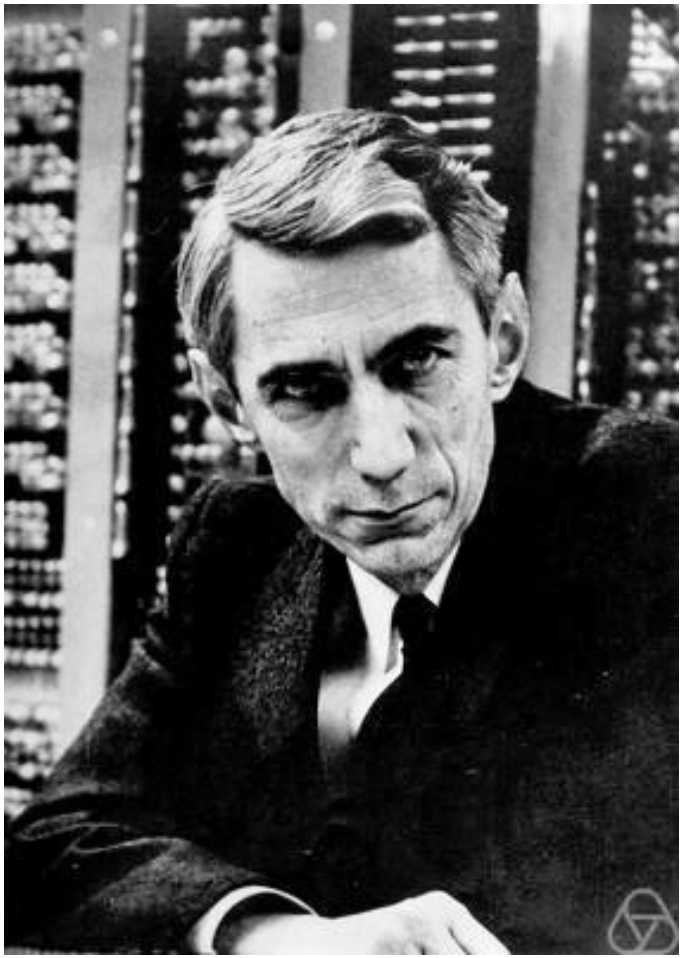
C_3 = {00000, 01101, 10111, 11011}

Transmission through Channel

- Physical medium through which information is transmitted (ex. Telephone lines, the atmosphere)
- Noise: may be caused by sunspots, lighting, meteor showers, random radio disturbance, poor typing, poor hearing, ...
- Transmission Goals
  - Fast encoding of information
  - Easy transmission of encoded messages
  - Fast decoding of received messages
  - Reliable correction of errors introduced in the channel
  - Maximum transfer of information per unit time
- BASIC METHOD OF FIGHTING ERRORS: REDUNDANCY!!!

  0 is encoded as 00000 and 1 is encoded as 11111

Claude Elwood Shannon

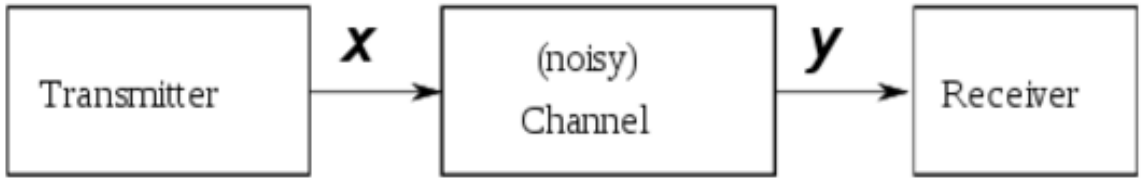


“The father of Information Theory”

American mathematician, electrical engineer, and cryptographer

(Apr.30, 1916 – Feb.24, 2001)

Shannon's Theorem



- Noisy-channel coding theorem
- Given a noisy channel with channel capacity *C* and the information transmitted at the rate *R*
  - If *R* < *C*, it is possible to transmit information nearly without error at any rate below a limiting rate
  - If *R* > *C*, an arbitrarily small probability of error is not achievable
- *Channel capacity: the highest information rate that can be achieved with arbitrarily small error probability*

$$C = B \log_2(1 + \frac{S}{N})$$

Channel Capacity

- Channel Capacity *C*: Probabilistic properties of the channel
- Encoding messages of *b* bits into code words of *b/R* bits, for some rate *0 < R < 1*
- *Error* is the event that the original message cannot be correctly decoded from the received code word
- *R* is achievable using a channel, if there is an encoding such that the probability of *error* goes to *zero* as *b increases*

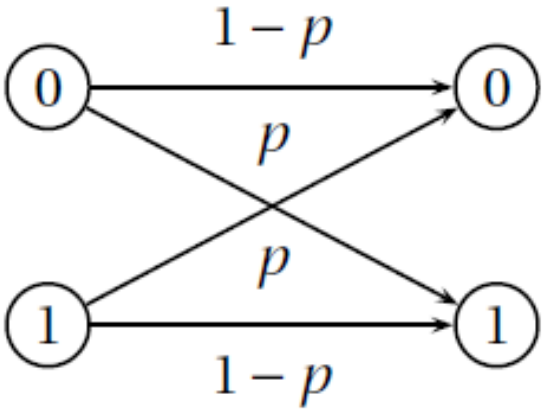
- The Source Coding (Shannon's 2nd Theorem):

  Rate R is achievable if R < C,

  and not achievable if R > C

Binary Symmetric Channel

- The probability of error $p < ½$
- If n symbols are transmitted,



then the probability of t errors is

$$p^t(1-p)^{n-t}\binom{n}{t}$$

- Consider $C = \{000,111\}$,

$$P_{err}(C) = 1 - ((1-p)^3 + 3p(1-p)^2)$$

- If p = 0.01, then $P_{\text{err}}(C)$ = 0.000298 and only one word in 3555 will reach the user with an error

Binary symmetric channel (BSC) capacity

- Error rate p:

$$\Pr[y = 1|x = 0] = \Pr[y = 0|x = 1] = p$$

where x is the transmitted and y the received bit

- Channel capacity C(p):

$$C(p) = 1 - H(p) = 1 - [p \log_2 \frac{1}{p} + (1 - p) \log_2 \frac{1}{1-p}]$$

(*H*: binary entropy function)

- For instance, $C(0.1) \approx 0.53$, *Ratio about 1:2*

Channel Coding Theorem, … So what?

- Assume you want to transmit data with probability of error 10^{-15} over a BSC, p = 0.1
- Using a *repetition code*, we need to make the message *63 times* as long as the source string (binomial distribution)

- Shannon's result says *twice* as long is enough!
- If you want probability of error 10^{-100}, Shannon's result still says that *twice* is enough!!
- However the messages you encode need to be sufficiently long!

Example

The encoding

$$0 \rightarrow 000 \qquad 1 \rightarrow 111$$

The probability of the bit error $p \leq \frac{1}{2}$, and the majority voting decoding

$$000, 001, 010, 100 \rightarrow 000, \qquad 111, 110, 101, 011 \rightarrow 111$$

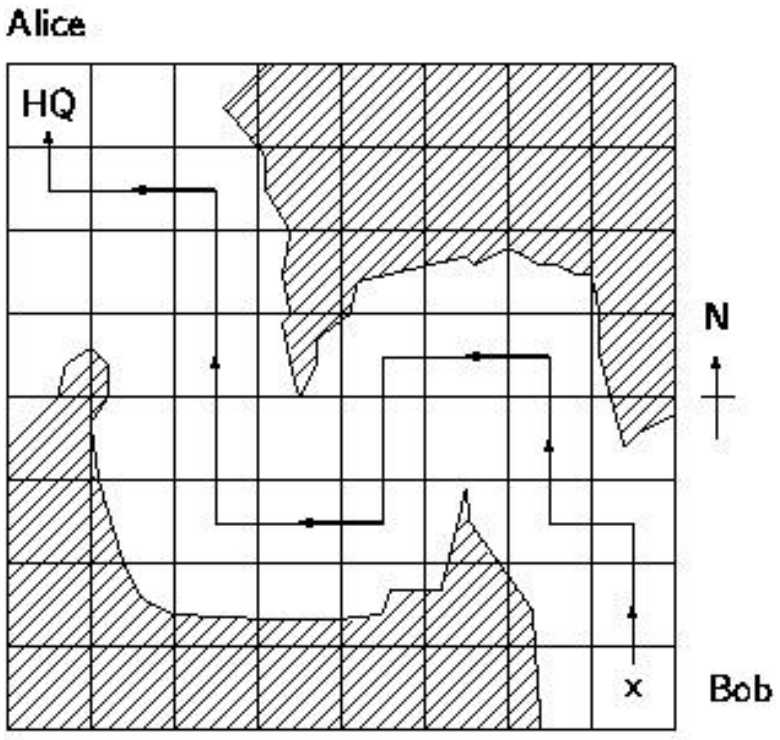
The probability of an erroneous message is

$$3p^2(1-p) + p^3 = 3p^2 - 2p^3 < p$$

Coding of a path avoiding an enemy territory

Story: Alice and Bob share an identical map gridded as shown. Only Alice knows the route through which Bob can reach her avoiding the enemy territory. Alice wants to send Bob the following information about the safe route he should take

NNWNNWWSSWWNNNNWWN



NNWNNWWSSWWNNNNWWN

Three ways to encode

- $C_1 = \{00, 01, 10, 11\}$
  - 000001000001011111010100000000010100
  - Any error in the code word would be a disaster
- $C_2 = \{000, 011, 101, 110\}$
  - A single error in encoding each of N, W, S, E could be detected
- $C_3 = \{00000, 01101, 10110, 11011\}$
  - A single error in decoding each of N, W, S, E could be corrected

Definition of Code

- A code C over an alphabet Σ is a subset of $\Sigma^*(\mathrm{C} \subseteq \Sigma^*)$
- A *q*-nary code is a code over alphabet of *q* symbols
- A binary code is a code over the alphabet $\{0,1\}$

Richard Wesley Hamming

“Hamming Distance”
American mathematician
“The purpose of computing is insight, not numbers”
(Feb.11, 1915 – Jan.7, 1998)



Hamming Weight

- Hamming Weight $w(c)$*:*
  - the number of nonzero components in the codeword

ex.

$w(00000) = 0$

$w(11111) = 5$

$w(1022001) = 4$

$w(1011001) = 4$

Hamming Distance

- Hamming Distance $h(x, y)$*:*
  - The number of positions where words *x* and *y* differs

  ex: $h(10101;\ 01100) = 3,\ \ h(fourth;\ eighth) = 4$
- Properties:
  - $h(x, y) = 0 \Leftrightarrow x = y$
  - $h(x, y) = h(y, x)$
  - $h(x, z) \leq h(x, y) + h(y, z) \Rightarrow$ *triangle inequality*
- Minimal distance:

$$h(C) = \min\{h(x, y) | x, y \in C, x \neq y\}$$

Code C can detect up to s errors if $h(C) \geq s + 1$

Code C can correct up to t errors if $h(C) \geq 2t + 1$

Hamming VS Shannon

Hamming	Shannon
Focus on codewords itself	Directly deal wt encoding and decoding functions
Look at explicit codes	Not explicit at all
Fundamental trade off: rate vs distance (easy to handle)	Fundamental trade off: rate vs error
Worst case errors	Stochastic errors

Power of Parity Bits

- Let all 2^{11} of binary words of length 11 be codewords
- Let the probability of an error be 10^{-8}
- Let bits be transmitted at the rate 10^7 bits per second

- The probability that a word is transmitted incorrectly is approximately
$$11p(1-p)^{10} \approx \frac{11}{10^8}.$$
- Therefore $\frac{11}{10^8} \cdot \frac{10^7}{11} = 0.1$ of words per second (every 10 seconds) are transmitted incorrectly

- Let one parity bit be added $\Rightarrow$ Any single error can be detected
- The probability of at least two errors is:
$$1-(1-p)^{12} - 12(1-p)^{11} p \approx \binom{12}{2}(1-p)^{10} p^2 \approx \frac{66}{10^{16}}$$
- Therefore approximately $\frac{66}{10^{16}} \cdot \frac{10^7}{12} \approx 5.5 \cdot 10^{-9}$ words per second are transmitted with an undetectable error
- **Corollary** One undetected error occurs only every 2000 days! ($2000 \approx 10^9/(5.5 \times 86400)$.)

Two-Dimensional Parity Code

- The two-dimensional parity code arranges the data into a two-dimensional array and then to each row (column) parity bit is attached

ex. 10001011000100101111

$$\begin{matrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{matrix} \rightarrow \begin{matrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 \end{matrix}$$

Notation

An (n,M,d) - code C is a code such that

- n - is the length of codeword
- M - is the number of codeword
- d - is the minimum distance in C

ex.

C_1 = {00, 01, 10, 11} is a (2,4,1) - code

C_2 = {000, 011, 101, 110} is a (3,4,2) - code

C_3 = {00000, 01101, 10110, 11011} is a (5,4,3) - code

A good (n,M,d) code has small n and large M and d

Optimization

A good (n,M,d) -code has small n, large M and large d

Notation: $A_q(n,d)$ is the largest M such that there is an q-nary (n,M,d)-code

Theorem: (a) $A_q(n,1) = q^n$

(b) $A_q(n,n) = q$

ex. $A_2(8, d)$ for $d = 1$ and $d = 2$

1. $d = 1$. $A_2(8, 1) = 2^8$. Code C contains all binary words of the length eight
2. $d = 2$. $A_2(8, 2) = A_2(7, 1) = 2^7$. Code C contains all binary words of the length seven with the parity bit added

Design of one code from another code

- If d is odd, then $A_2(n,d) = A_2(n+1,d+1)$
- If d is even, then $A_2(n,d) = A_2(n-1,d-1)$

ex. $A_2(5,3) = 4 \Rightarrow A_2(6,4) = 4$

$(5,4,3)\text{-code} \Rightarrow (6,4,4)\text{-code}$

0	0	0	0	0
0	1	1	0	1
1	0	1	1	0
1	1	0	1	1

by adding check

Equivalence

- Two q-nary codes are equivalent if one can be obtained from the other by a combination of following operations:

1. permutation of the positions of the code
2. permutation of symbols at the fixed positions

$$(1)\ \begin{Bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \end{Bmatrix} \begin{Bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 \end{Bmatrix} \quad (2)\ \begin{Bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{Bmatrix} \begin{Bmatrix} 0 & 1 & 2 \\ 1 & 2 & 0 \\ 2 & 0 & 1 \end{Bmatrix}$$

Linear Codes

- Binary Linear Codes
  - A *binary linear code C* of length *n* is a set of binary *n*-tuples such that the componentwise modulo 2 sum of any two codewords is contained in *C*

ex.5 (4,4,2)-code: {0000,0101,1010,1111}

(7,16,3)-code: { ?? }

{0000000,0001111,0010110,0011001,0100101,0101010
,0110011,0111100,1000011,1001100,1010101,1011010,1100110,110
1001,1110000,1111111}

Cyclic Codes

A code *C* is cyclic if

1. *C* is a linear code
2. any cyclic shift of a codeword is also a codeword

ex1. The ternary code C = {0000,1212,2121} is cyclic

1. Linear: c1+c2=c2, c1+c3=c3, c2+c3=c1, c2+c2=c3, c3+c3=c2
2. Cyclic: c1 $\rightarrow$ c1, c2 $\rightarrow$ c3 $\rightarrow$ c2

ex2. The binary code C = {000,110,011,101} is cyclic

ex3. 5-bit cyclic code C = {00000,00011,00110,01100,11000,10001, 00101,01010,10100,01001,10010,01111,11110,11101,11011,10111}

Check yourself 1

- Use nearest neighbor decoding to determine which code word was sent from the code C = {0000, 1110, 1011} if 0110 is received?

d(0110,0000)=?

d(0110,1110)=?

d(0110,1011)=?

Check yourself 2

- Suppose that when a bit is sent over a binary symmetric channel the probability it is received incorrectly is 0.01. What is the probability that the bit string 100010 is received when the bit string 000000 is sent?

  Maximum likelihood decoding:

  Prob (4 correct & 2 incorrect) =

-

Check yourself 3

- Find the minimum distance of the code;
  C = {00000, 01110, 10011, 11111}

All combinations

d(00000, 01110) = ?
d(00000, 10011) = ?
d(00000, 11111) = ?
d(01110, 10011) = ?
d(01110, 11111) = ?
d(10011, 11111) = ?

Check yourself 4

- Let C = {00000000, 11111000, 01010111, 10101111},

How many errors can C detect and how many can it correct?

Minimum distance C {00000000, …, 10101111} = ?

Detect upto = (minimum distance – 1) = ?

Correct upto = | (minimum distance – 1) / 2 | = ?

Cyclic Codes - Theory

- k - number of bits of data that are encoded
- Encoded word of length n bits
  - obtained by multiplying the given k data bits by a number that is n-k+1 bits long
- The multiplier is represented as a polynomial
  - the generator polynomial
- 1s and 0s in the n-k+1-bit multiplier are treated as coefficients of an (n-k) -degree polynomial
- Example: multiplier is 11001 - generator polynomial is

$$G(x)=1X^0 + 0X^1 + 0X^2 + 1X^3 + 1X^4 = 1 + X^3 + X^4$$

Cyclic codes

- **Definition:** Any cyclic shift of a codeword produces another codeword
- A *k*-bit data word corresponds to a polynomial of degree *k-1*

  Data = 1101001: $d(x) = 1 + x + x^3 + x^6$ (modulo-2 sum)

  (n, k) cyclic code has a generator polynomial of degree $r = n-k$

  $g(x) = 1 + x + x^3$

  To encode data (1101001), multiply its associated polynomial by $g(x)$

$$
\begin{array}{l}
\quad 1 + x + x^3 + x^6 \\
\times \ \underline{1 + x + x^3} \\
\quad 1 + \cancel{x} + \cancel{x^3} + \cancel{x^6} + \cancel{x} + x^2 + \cancel{x^4} + x^7 + \cancel{x^3} + \cancel{x^4} + \cancel{x^6} + x^9 \\
\quad 1 + x^2 \quad + \quad x^7 + x^9 \\
\quad 1\ 0\ 1\ 0\ 0\ 0\ 1\ 0\ 1
\end{array}
$$

- Detect all burst errors of width less than *n-k*

An (n,k) Cyclic Code

- Using a generator polynomial of degree n-k and total number of encoded bits n
- An (n,k) cyclic code can detect all single errors and all runs of adjacent bit errors shorter than n-k
- For a polynomial to be a generator polynomial for an (n,k) cyclic code it must be a factor of X^n-1 ($=X^n+1$)
- $1+X^3+X^4$ is a factor of $X^{15}-1 \rightarrow$ (15,11) code

  $X^{15}-1=(X+1)(X^2+X+1)(X^4+X+1)(X^4+X^3+1)(X^4+X^3+X^2+X+1)$
- For the 5-bit code, (X+1) – generator polynomial

  $X^5-1 = (X+1)(X^4+X^3+X^2+X+1)$
- Multiply {0000,….,1111} by (X+1) to obtain all codewords of the (5,4) code

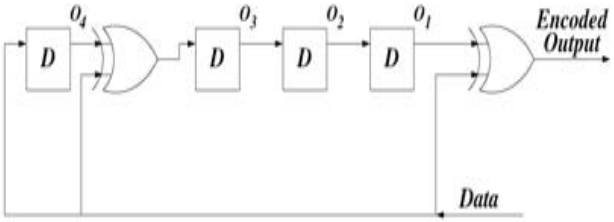
Factorization of X^n-1 (X^n+1)

n	$x^n + 1 =$
1	$(x+1)$
2	$(x+1)^2$
3	$(x+1)(x^2+x+1)$
4	$(x+1)^4$
5	$(x+1)(x^4+x^3+x^2+x+1)$
6	$(x+1)^2(x^2+x+1)^2$
7	$(x+1)(x^3+x+1)(x^3+x^2+1)$
8	$(x+1)^8$
9	$(x+1)(x^2+x+1)(x^6+x^3+1)$
10	$(x+1)^2(x^4+x^3+x^2+x+1)^2$
11	$(x+1)(x^{10}+x^9+x^8+x^7+x^6+x^5+x^4+x^3+x^2+x+1)$
12	$(x+1)^4(x^2+x+1)^4$
13	$(x+1)(x^{12}+x^{11}+x^{10}+x^9+x^8+x^7+x^6+x^5+x^4+x^3+x^2+x+1)$
14	$(x+1)^2(x^3+x+1)^2(x^3+x^2+1)^2$
15	$(x+1)(x^2+x+1)(x^4+x+1)(x^4+x^3+1)(x^4+x^3+x^2+x+1)$
16	$(x+1)^{16}$
17	$(x+1)(x^8+x^5+x^4+x^3+1)(x^8+x^7+x^6+x^4+x^2+x+1)$
18	$(x+1)^2(x^2+x+1)^2(x^6+x^3+1)^2$
19	$(x+1)(x^{18}+x^{17}+x^{16}+\cdots+x+1)$
20	$(x+1)^4(x^4+x^3+x^2+x+1)^4$
21	$(x+1)(x^2+x+1)(x^3+x+1)(x^3+x^2+1)(x^6+x^4+x^2+x+1)(x^6+x^5+x^4+x^2+1)$
22	$(x+1)^2(x^{10}+x^9+x^8+x^7+x^6+x^5+x^4+x^3+x^2+x+1)^2$
23	$(x+1)(x^{11}+x^9+x^7+x^6+x^5+x+1)(x^{11}+x^{10}+x^6+x^5+x^4+x^2+1)$
24	$(x+1)^8(x^2+x+1)^8$
25	$(x+1)(x^4+x^3+x^2+x+1)(x^{20}+x^{15}+x^{10}+x^5+1)$
26	$(x+1)^2(x^{12}+x^{11}+x^{10}+x^9+x^8+x^7+x^6+x^5+x^4+x^3+x^2+x+1)^2$
27	$(x+1)(x^2+x+1)(x^6+x^3+1)(x^{18}+x^9+1)$
28	$(x+1)^4(x^3+x+1)^4(x^3+x^2+1)^4$
29	$(x+1)(x^{28}+x^{27}+\cdots+x+1)$
30	$(x+1)^2(x^2+x+1)^2(x^4+x+1)^2(x^4+x^3+1)^2(x^4+x^3+x^2+x+1)^2$
31	$(x+1)(x^5+x^2+1)(x^5+x^3+1)(x^5+x^3+x^2+x+1)$ $\cdot(x^5+x^4+x^2+x+1)(x^5+x^4+x^3+x+1)(x^5+x^4+x^3+x^2+1)$

Modulo-2 Multiplication

- The 5th bit of the product is the modulo-2 sum of the corresponding bits of the multiplicand shifted 0 times, 3 times, and 4 times
- If multiplicand is fed in serially – we add the shifted multiplicand
- Shifting done by delay elements
- This cyclic code is not separable - data and check bits within 110000100011101 are not separable

$$
\begin{array}{r}
10001100101 \\
\times \quad 11001 \\
\hline
10001100101 \\
00000000000 \\
00000000000 \\
10001100101 \\
10001100101 \\
\hline
11000010001101
\end{array}
$$

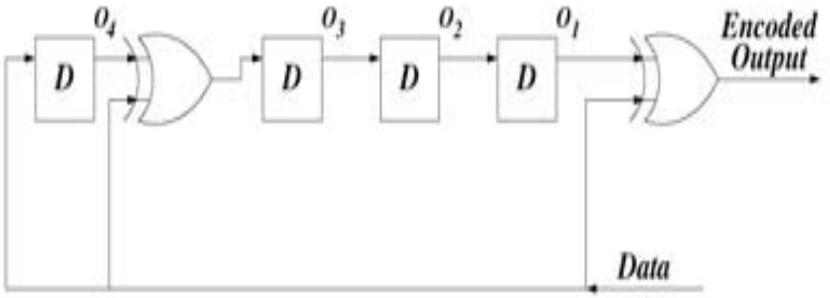


Operation of Encoding Code

shift clock	input data	O_4	i_3	O_3, O_2, O_1	encoded output
1	1	0	1	000	1
2	0	1	1	100	0
3	1	0	1	110	1
4	0	1	1	111	1
5	0	0	0	111	1
6	1	0	1	011	0
7	1	1	0	101	0
8	0	1	1	010	0
9	0	0	0	101	1
10	0	0	0	010	0
11	1	0	1	001	0
12	0	1	1	100	0
13	0	0	0	110	0
14	0	0	0	011	1
15	0	0	0	001	1

- i3 is the input to the O3 delay element

110000100011101



10001100101

Decoding Through Division by Generator Polynomial

```
1100001000111101 : 11001 = 10001100101
11001
 10100
 11001
  11010
  11001
   11111
   11001
    11001
    11001
    00000
```

```
1100001011101 : 11001 = 10001100110
11001
 10100
 11001
  11011
  11001
   10111
   11001
    11100
    11001
    01011
```

error!!

- Division by 11001
- Subtraction modulo-2 - identical to addition
- Zero remainder indicates no error detected
- If a single error occurs – 1100001001111101 - non-zero remainder

Three Bit Errors

$$
\begin{array}{l}
1100001\mathbf{11}0\mathbf{1}0101 : 11001 = 10001101101 \\
11001 \\
\hline
\quad 10111 \\
\quad 11001 \\
\quad \hline
\quad\ 11100 \\
\quad\ 11001 \\
\quad\ \hline
\quad\quad 10110 \\
\quad\quad 11001 \\
\quad\quad \hline
\quad\quad\ 11111 \\
\quad\quad\ 11001 \\
\quad\quad\ \hline
\quad\quad\quad 11001 \\
\quad\quad\quad 11001 \\
\quad\quad\quad \hline
\quad\quad\quad\ 00000
\end{array}
$$

$$
\begin{array}{l}
1100000\mathbf{11}011101 : 11001 = 10001110011 \\
11001 \\
\hline
\quad 10011 \\
\quad 11001 \\
\quad \hline
\quad\ 10100 \\
\quad\ 11001 \\
\quad\ \hline
\quad\quad 11011 \\
\quad\quad 11001 \\
\quad\quad \hline
\quad\quad\ 10110 \\
\quad\quad\ 11001 \\
\quad\quad\ \hline
\quad\quad\quad 11111 \\
\quad\quad\quad 11001 \\
\quad\quad\quad \hline
\quad\quad\quad\ 00110
\end{array}
$$

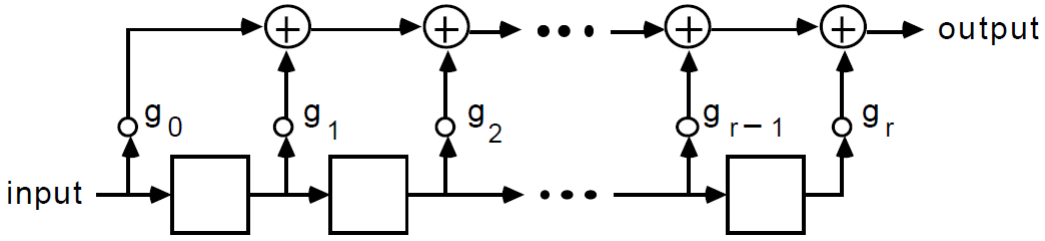
- 1100001110101 01 instead of 1100000100011101 - zero remainder - errors not detected (erroneously)
- If errors are adjacent - 1100000**11**011101 - non-zero remainder indicates an error

Cyclic code implementation

- Multiply *K* bit data by *(n-k+1)* bit multiplier (*(n-k)* degree polynomial), the result is *n* bit encoded data: *(n,k) cyclic code*
- Shift register circuit for multiplying by generator polynomial,

$$g(x) = g0+g1x+\cdots+grxr$$

- An encoder for an (*n, k*) cyclic code with generator polynomial *g*(*x*), with *r = n-k*



Implementing a Divider Circuit

- Division can be done by multiplication in the feedback loop
- Example: Encoded word - polynomial $E(X)$, generator polynomial - $G(X)$, original data word - polynomial $D(X)$
- If no bit errors exist - we will receive $E(X)$ and calculate $D(X)$ by $D(X)=E(X)/G(X)$ - remainder will be zero
- Example: $E(X)=D(X)G(X)=D(X)(1+X^3+X^4)$

  $= D(X)+D(X)(X^3+X^4)$
- $D(X)=-E(X)+D(X)(X^3+X^4)=E(X)+D(X)(X^3+X^4)$

* (since addition = subtraction in mod-2 arithmetic)

Generator Polynomial

- (n,k) cyclic code:

- If n = 5: $x^5 + 1 = (x + 1)(x^4 + x^3 + x^2 + x + 1)$

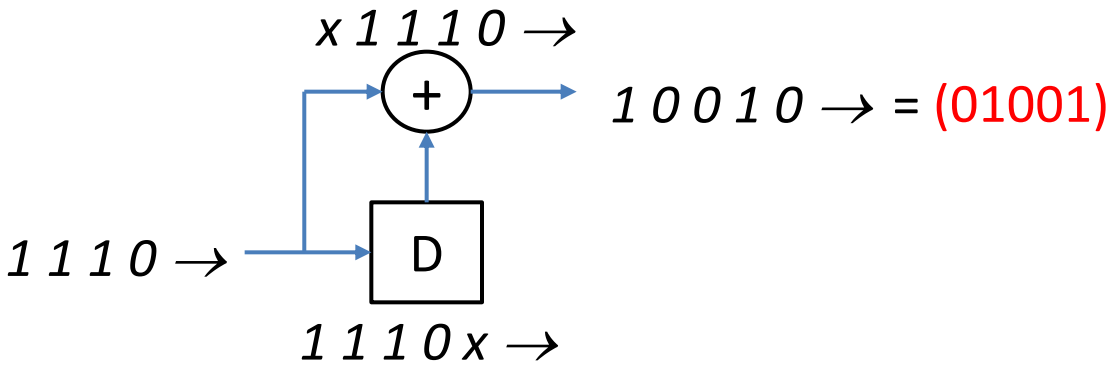
  example 1: k = 4, $g(x) = (x + 1)$

- If n = 7: $x^7 + 1 = (x + 1)(x^3 + x + 1)(x^3 + x^2 + 1)$

  example 2: k = 4, $g(x) = (x^3 + x + 1)$

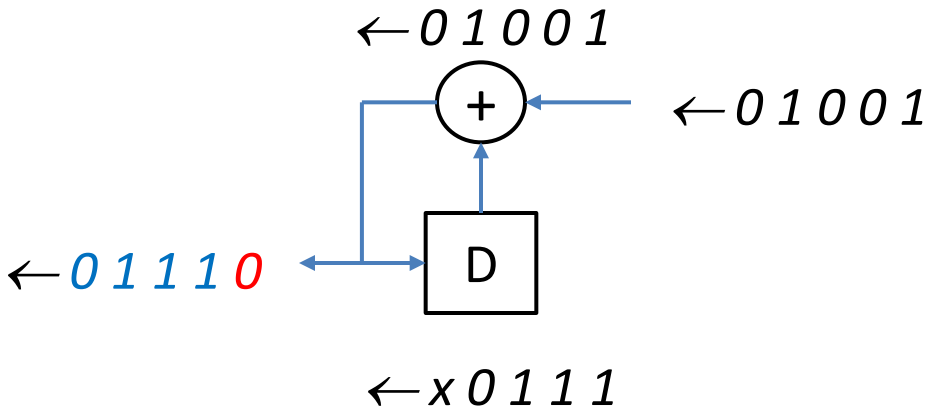
Cyclic code encoding example 1

- k = (0111), g(x) = (1 + x)



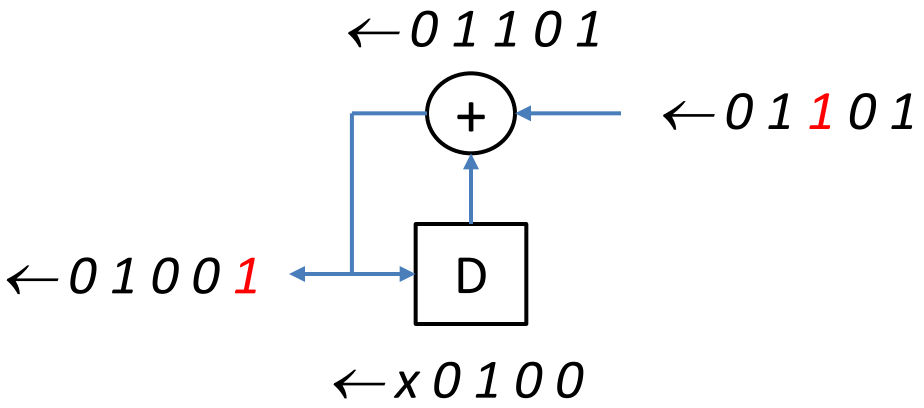
Cyclic code decoding example 1

- k(x) = e(x)/g(x) – Remainder (zero if no error)
- E = (01001), g(x) = (1 + x) $\Rightarrow$ k(x) = e(x) + k(x)*x



Cyclic code decoding example 1

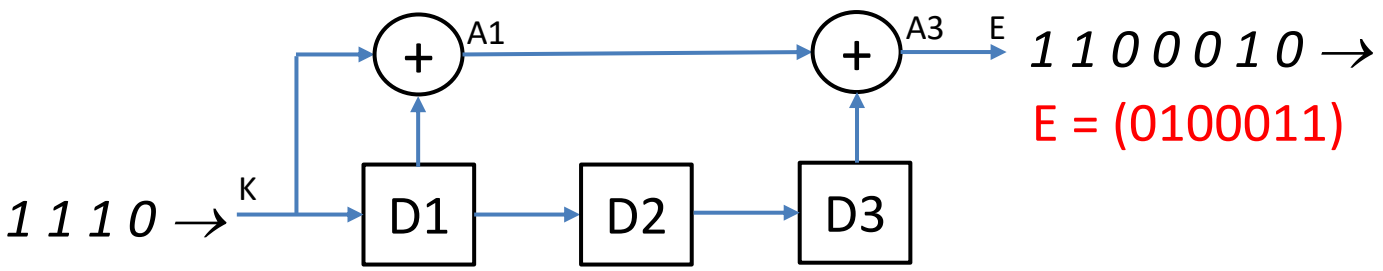
- K(X) = E(X)/G(X) – Remainder (non-zero if error)
- E = (01101), g(x) = (1 + x) $\Rightarrow$ K(X) = E(X) + K(X)*X



Cyclic code encoding example 2

- K = (0111), $g(x) = (1 + x + x^3)$

```
    0111
    1101
    0111
  0111
0111
0100011
```

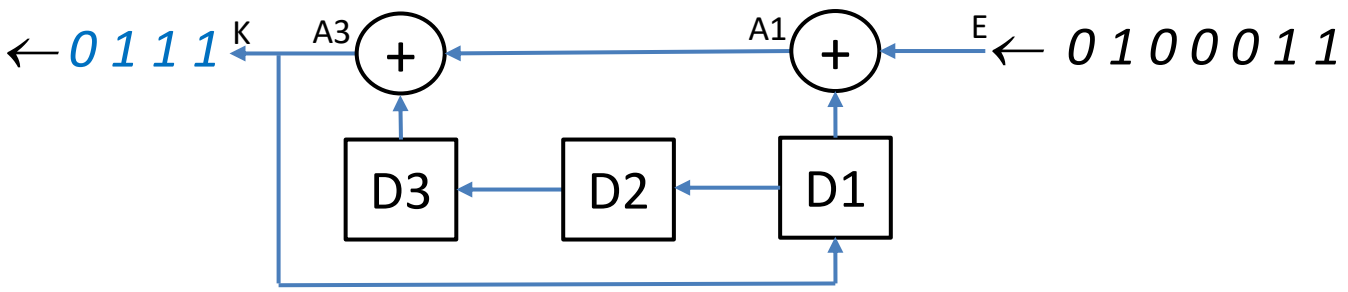


K	D1	A1	D2	D3	A3 (E)
0	x	0	x	x	0
1	0	1	x	x	1
1	1	0	0	x	0
1	1	0	1	0	0
0	1	1	1	1	0
0	0	0	1	1	1
0	0	0	0	1	1

Cyclic code decoding example 2

- E = (0100011), $g(x) = (1 + x + x^3) \Rightarrow K(X) = E(X) + K(X)(X + X^3)$

```
0100011
 1101    :01
  10111
  1101   :1
   1101
   1101  :1
    000
```

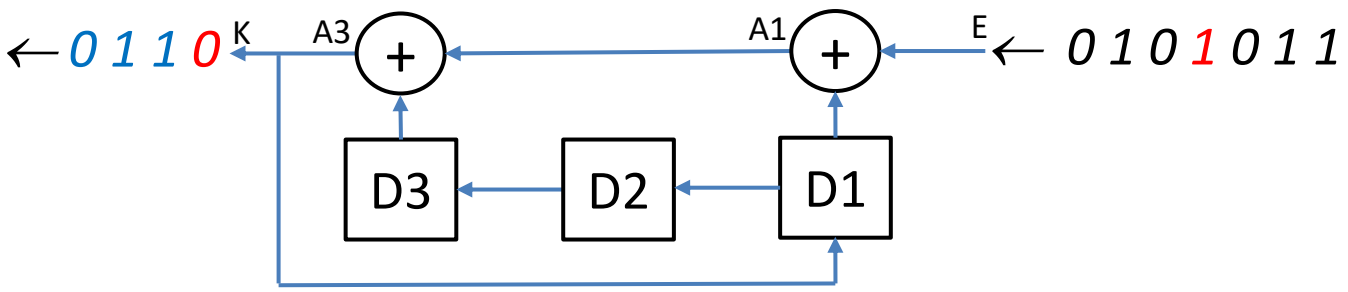


E	D1	D2	D3	A1	A3 (K)
0	x	x	x	0	0
1	0	x	x	1	1
0	1	0	x	1	1
0	1	1	0	1	1
0	1	1	1	1	0
1	0	1	1	1	0
1	0	0	1	1	0

Cyclic code decoding example 2

- E = (0101011), $g(x) = (1 + x + x^3) \Rightarrow K(X) = E(X) + K(X)(X + X^3)$

```
0101011
 1101    :01
  11111
  1101   :1
   0101
   0000  :0
    101
```



E	D1	D2	D3	A1	A3 (K)
0	x	x	x	0	0
1	0	x	x	1	1
0	1	0	x	1	1
1	1	1	0	0	0
0	0	1	1	0	1
1	1	0	1	0	1
1	1	0	0	0	0

Code Rate

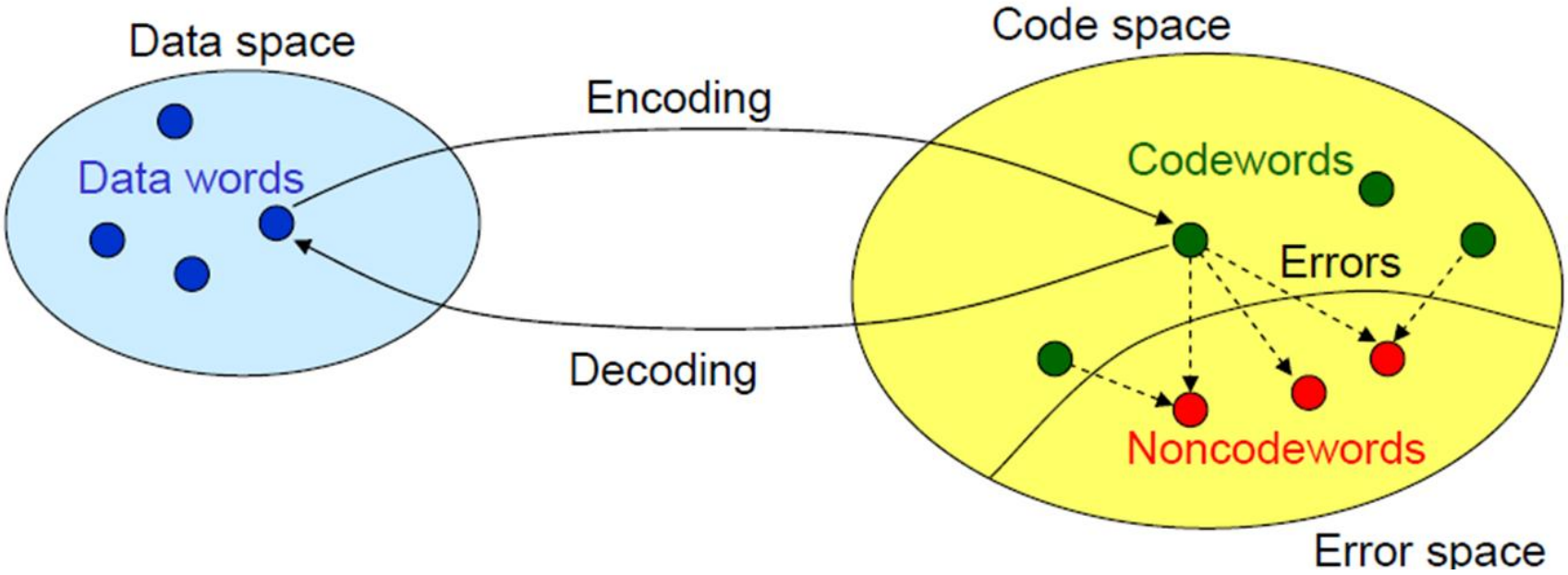
- For q-nary (n, M, d) - code we define code rate, or information rate, R, by

$$R = \frac{\log_q M}{n}$$

- The code rate represents the ratio of the number of needed input data symbols to the number of transmitted code symbols

- Code rate is an important parameter for real implementations, because it shows what fraction of the bandwidth is being used to transmit actual data

Error Detection



Error Detection

- Error detection is much more modest aim than error correction
- Error detection is suitable in the cases that channel is so good that probability of error is small and if an error is detected, the receiver can ask to renew the transmission
- For example, two main requirements for many telegraphy codes used to be:
  - Any two codewords had to have distance at least 2;
  - No codeword could be obtained from another codeword by transposition of two adjacent letters

Evaluation of error-detecting codes

- **Redundancy:** *k* data bits encoded in *n* = *k*+*r* bits (*r* redundant bits)
- **Encoding:** Complexity (cost / time) to form codeword from data word
- **Decoding:** Complexity (cost / time) to obtain data word from codeword

  Separable codes have computation-free decoding
- **Capability:** Classes of error that can be detected

  Greater detection capability generally involves more redundancy

  To detect *d* bit-errors, a minimum code distance of *d*+ 1 is required

  Examples of code detection capabilities:

  Single, double, *b*-bit burst, byte, unidirectional, . . . errors
- **Closure:** Arithmetic and other operations done directly on codewords

  (rather than in 3 stages: decode, operate, and encode)

Further Readings *(Out-of-scope)*

- Other Properties of Codes
- ISBN Code
- Data Compression
  - ZIP Code, Reed-Solomon Code
- Cryptography
- Information Theory

HW05: Cyclic code

- Generate (7,4) Cyclic code from the data (0101, 1001, 0010, 1110, 0000), and its hardware implementation.

Home Exercise: Cyclic code 1/4 (calculation)

- Generate (7,4) Cyclic code from the data (0101, 1001, 0010, 1110, 0000), and its hardware implementation.

Ans: when m = [0101]

(7,4) Cyclic code: N = 7, N – K = 3, then

$x^7 + 1 = (x+1)(x^3+x+1)(x^3+x^2+1)$

for m = [0101] $\Rightarrow$ $m(x) = x+x^3$

$c(x) = m(x)g(x) = (x+x^3)(x^3+x^2+1) = x+x^4+x^5+x^6$

Hence, c = [0100111]

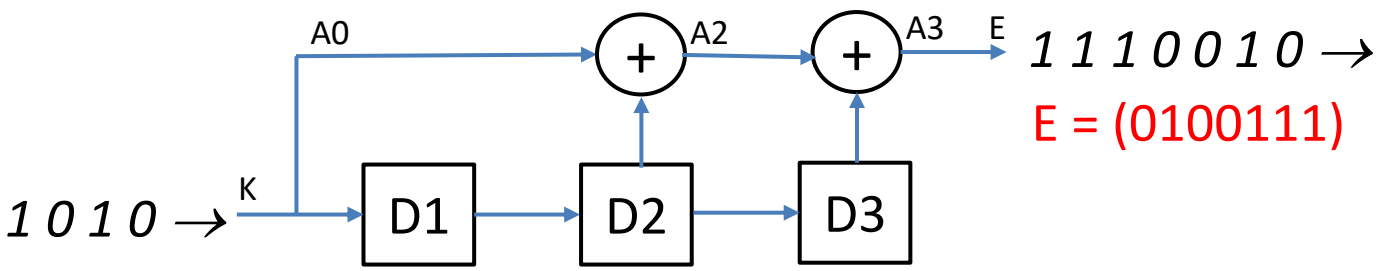
m(0101, 1001, 0010, 1110, 0000) $\Rightarrow$

c(0100111,1010011,0010110,1100010,0000000)

Home Exercise: Cyclic code 2/4 (Encoding)

- m = (0101), g(x) = (1 + x^2 + x^3)

```
    0101
    1011
    0101
   0101
0101
0100111
```



K (A0)	D1	D2	A2	D3	A3 (E)
0	x	x	0	x	0
1	0	x	1	x	1
0	1	0	0	x	0
1	0	1	0	0	0
0	1	0	0	1	1
0	0	1	1	0	1
0	0	0	0	1	1

Implementing a Divider Circuit

- Division can be done by multiplication in the feedback loop
- Example: Encoded word - polynomial E(X), generator polynomial - G(X), original data word - polynomial D(X)
- If no bit errors exist - we will receive E(X) and calculate D(X) by D(X)=E(X)/G(X) - remainder will be zero
- Example: $E(X)=D(X)G(X)=D(X)(1+X^3+X^4)$

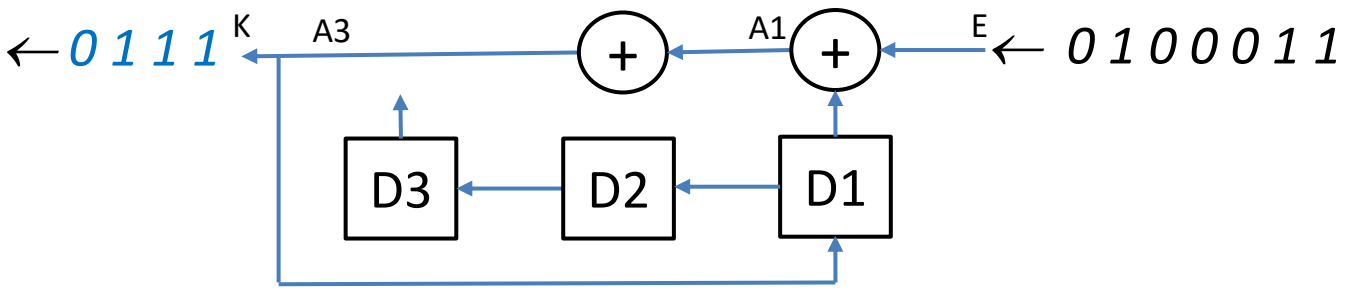
  $= D(X)+D(X)(X^3+X^4)$
- $D(X)=-E(X)+D(X)(X^3+X^4)=E(X)+D(X)(X^3+X^4)$

* (since addition = subtraction in mod-2 arithmetic)

Cyclic code decoding example 3/4

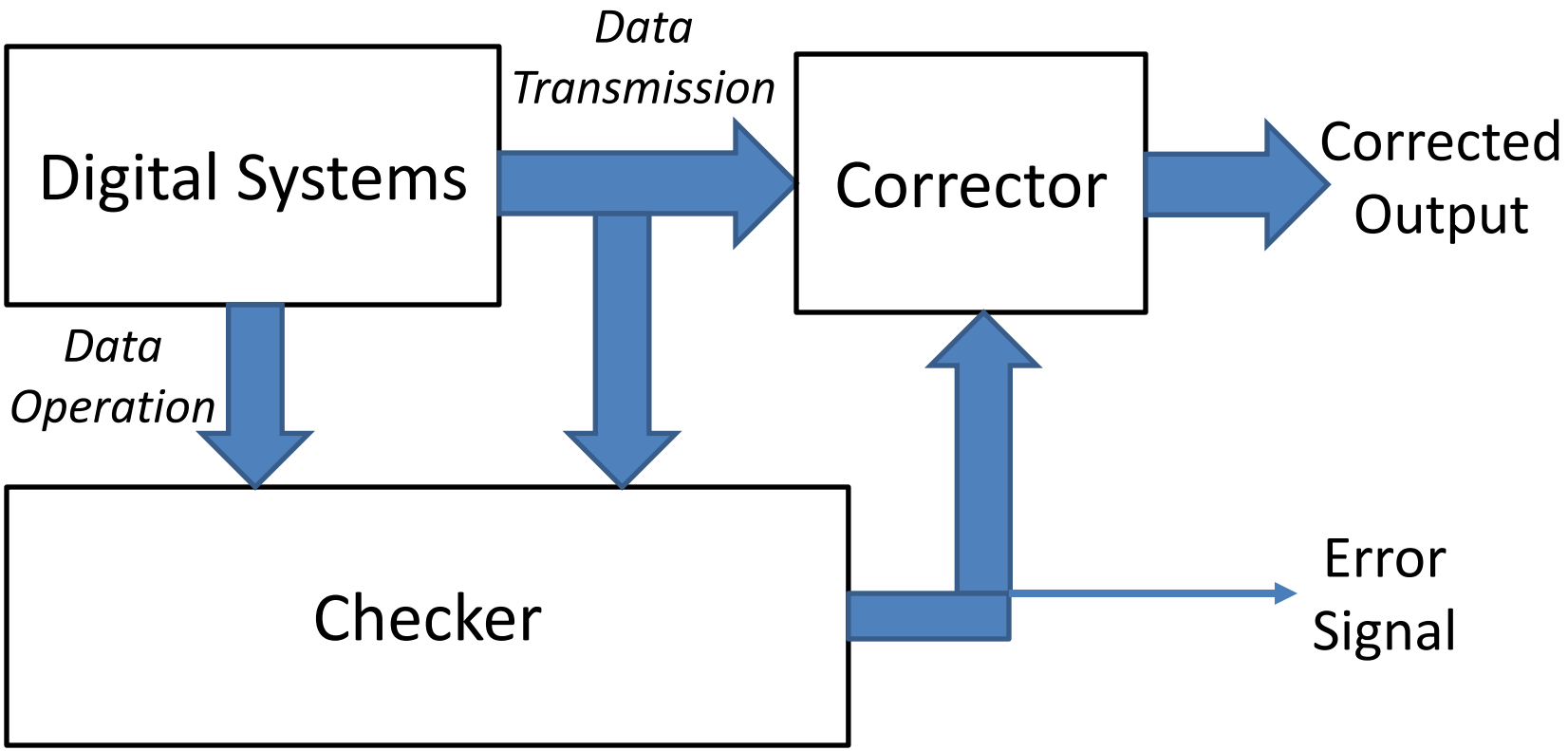
- E = (0100011), $g(x) = (1 + x^2 + x^3) \Rightarrow K(X) = E(X) + K(X)(X^2 + X^3)$

```
0100011
 1101    :01
  10111
  1101   :1
   1101
   1101  :1
    000
```



E	D1	D2	D3	A1	A3 (K)
0	x	x	x	0	0
1	0	x	x	1	1
0	1	0	x	1	1
0	1	1	0	1	1
0	1	1	1	1	0
1	0	1	1	1	0
1	0	0	1	1	0

Error Controls for Digital Systems



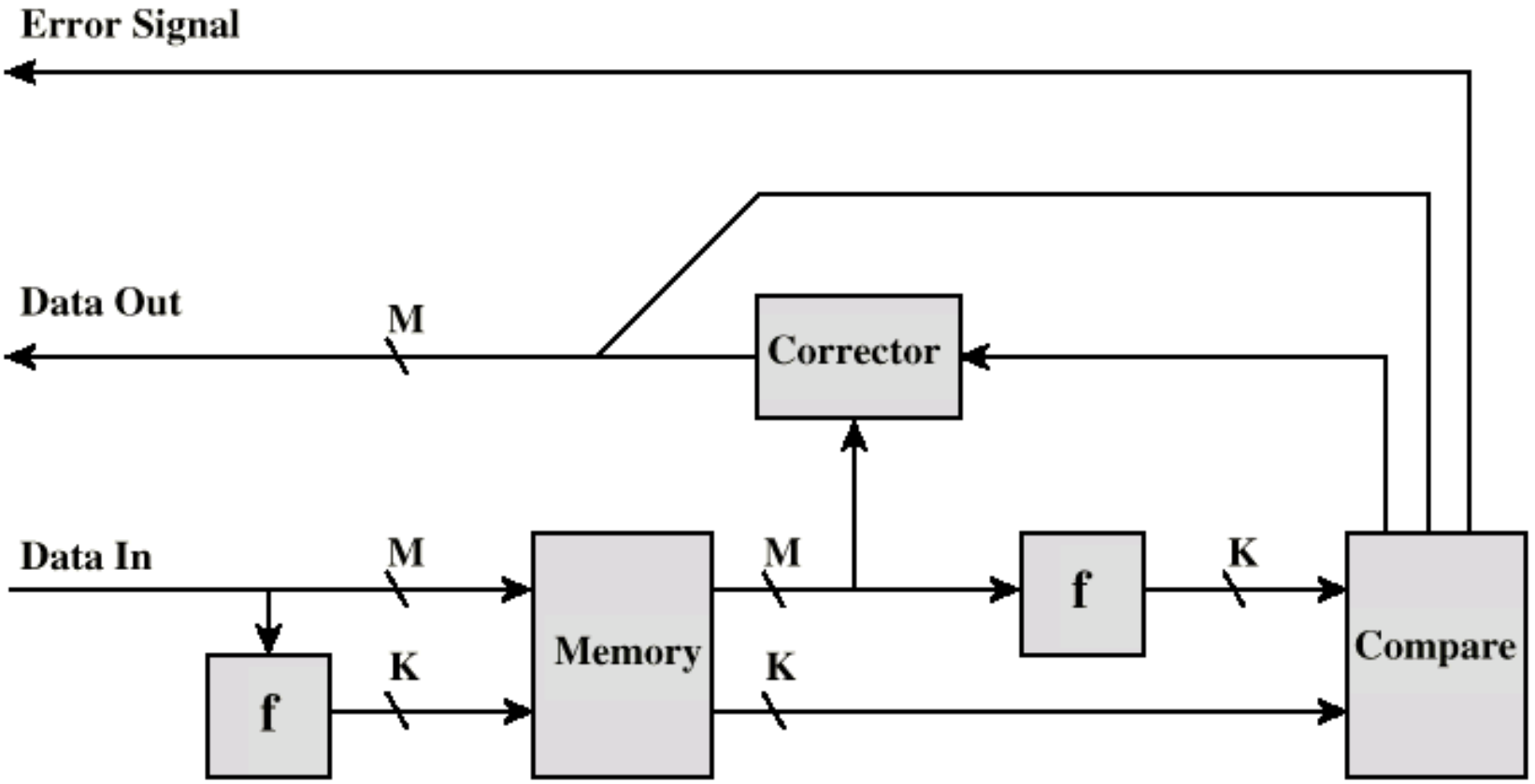
Error Model

- Symmetric / Asymmetric error
  - Both/Only one type of Known 0→1 and 1→0 errors can occur wt equal probability in a codeword
- Single / Multiple error
  - One/More than one Bit error
- Unidirectional error
  - Both Unknown 0→1 and 1→0 errors can occur, but they do not occur simultaneously in any codeword
- Byte error
  - Multiple error in K-Byte (K: constant)

Error Control Codes

- Error Correcting Code
  - Overlapped Parity Code
- Error Detection Code
  - Checksum Code
  - Residue Code
  - Unordered Code
    - m-of-n Code
    - Berger Code
    - Dual Rail Code

Error Correcting Code Function



Overlapped Parity Code

- Single Error Correcting (SEC)/ 8-Data bits wt 4-Check bits

12	11	10	9	8	7	6	5	4	3	2	1	Bit Position
0	1	0	1	0	1	0	1	0	1	0	1	
0	1	1	0	0	1	1	0	0	1	1	0	
1	0	0	0	0	1	1	1	1	0	0	0	
1	1	1	1	1	0	0	0	0	0	0	0	
				C4				C3		C2	C1	Check Bit
D8	D7	D6	D5		D4	D3	D2		D1			Data Bit

$$2^k - 1 \geq m + k$$

m = data bits
k = parity bits

Overlapped Parity Syndrome

- Syndrome is derived from comparing, i.e. XOR, transmitted and received/recomputed check bits
- Syndrome has following characteristics (previous example)
  - if syndrome contains all 0's
    - no error has been detected
  - if syndrome contains one and only one bit set to 1
    - error has occurred in one of the 4 check bits
  - if syndrome contains more than one bit set to 1
    - numerical value of the syndrome indicates the position of the data-bit error
    - this bit is then inverted for correction

Compute Check

12	11	10	9	8	7	6	5	4	3	2	1	Bit Position
0	1	0	1	0	1	0	1	0	1	0	1	
0	1	1	0	0	1	1	0	0	1	1	0	
1	0	0	0	0	1	1	1	1	0	0	0	
1	1	1	1	1	0	0	0	0	0	0	0	
				C4				C3		C2	C1	Check Bit
D8	D7	D6	D5		D4	D3	D2		D1			Data Bit

$$C1 = D1 \oplus D2 \oplus D4 \oplus D5 \oplus D7$$

$$C2 = D1 \oplus D3 \oplus D4 \oplus D6 \oplus D7$$

$$C3 = D2 \oplus D3 \oplus D4 \oplus D8$$

$$C4 = D5 \oplus D6 \oplus D7 \oplus D8$$

Example of Overlapped Parity

- Data sent is 1110 0001 and transmitted check bits are 1110
- Assume received data is: 01100001
- Received check bits are: 1110
- Recomputed check bits are: 0010
- Syndrome 1100⊕0010 = 1100

$$C1 = D1 \oplus D2 \oplus D4 \oplus D5 \oplus D7$$
$$C2 = D1 \oplus D3 \oplus D4 \oplus D6 \oplus D7$$
$$C3 = D2 \oplus D3 \oplus D4 \oplus D8$$
$$C4 = D5 \oplus D6 \oplus D7 \oplus D8$$

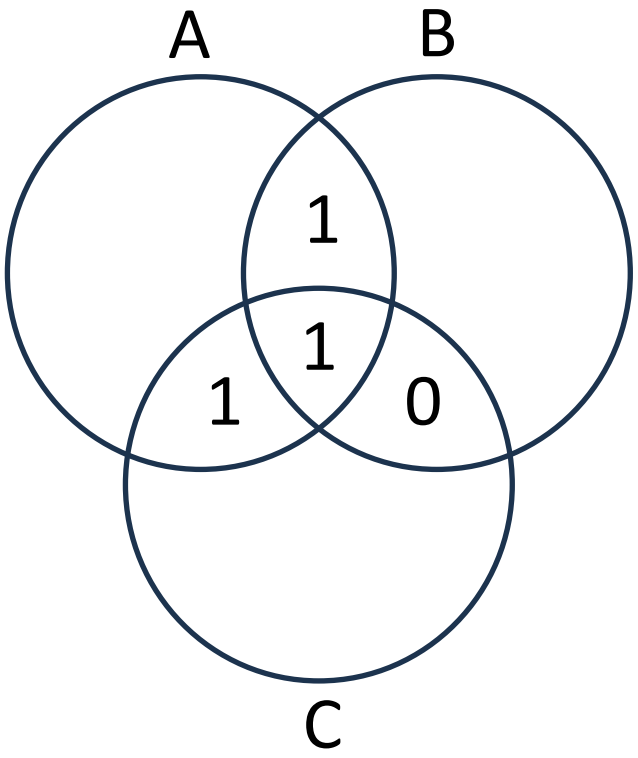
Applying Syndrome

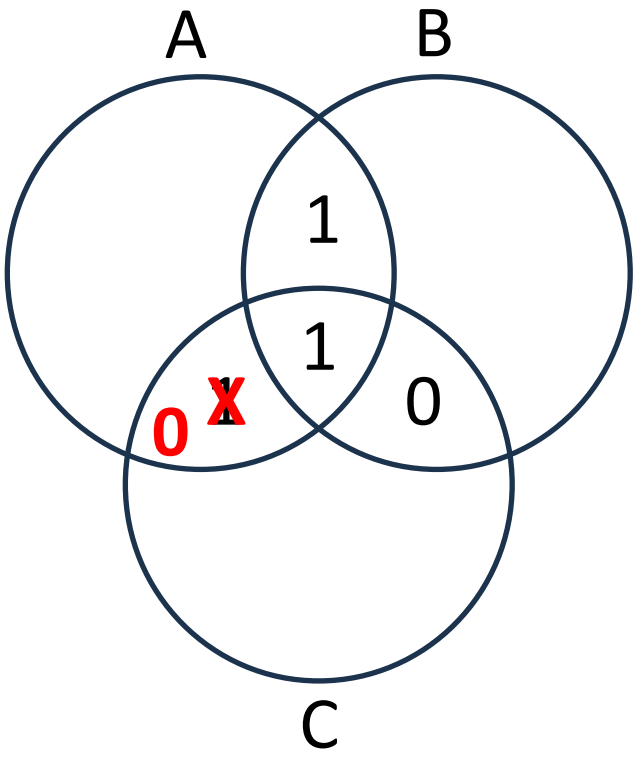
12	11	10	9	8	7	6	5	4	3	2	1	Bit Position
0	1	0	1	0	1	0	1	0	1	0	1	
0	1	1	0	0	1	1	0	0	1	1	0	
1	0	0	0	0	1	1	1	1	0	0	0	
1	1	1	1	1	0	0	0	0	0	0	0	
				C4				C3		C2	C1	Check Bit
D8	D7	D6	D5		D4	D3	D2		D1			Data Bit

Syndrome 1100 detects D8 as faulty

Check yourself 5

- ECC using Venn Diagram





BCH Code

- Bose–Chaudhuri–Hocquenghem codes (1959)
- A class of Cyclic error-correcting codes
- Using polynomials over a finite field (also called *Galois field*)
- Possible to design binary BCH codes that can correct multiple bit errors
- BCH codes is easy to be decoded, namely, via an algebraic method known as syndrome decoding
- Widely used in applications such as satellite communications, compact disc players, DVDs, disk drives, solid-state drives and two-dimensional bar codes

Checksum Code

- Separable code to achieve error detection capability
- Checksum is the sum of the original data
- Single-precision checksum
  - $C = x_1 + x_2 + \cdots x_n$
  - overflow problem, i.e. adding n bits modulo-2^n
- Double-precision checksum
  - use double precision, i.e. compute 2n-bit checksum from n-bit words using modulo-2^{2n} arithmetic
- Residue checksum
  - like single-precision checksum, but overflow is now fed back as carry
- Honeywell checksum
  - compose word of double length by concatenating 2 consecutive words
  - compute checksum on these double words

Checksum Code

- Single precision checksum: modulo-2^d addition
- Double precision checksum: modulo-2^{2d} addition
- Residue checksum: end-around carry addition
- Honeywell checksum: pair checksum

example:	SP	DP	RC	HC
	0000	0000	0000	
	0101	0101	0101	
	1111	1111	1111	00000101
	0010	0010	0010	11110010
	0110	00010110	0111	11110111

Checksum Code Example

- 16 bits block is to be sent using 8 bits checksum
- 10101001 00111001

Addition	10101001	
	00111001	
Sum	11100010	
Checksum	**00011101**	*(complement)*

- The pattern sent is 10101001 00111001 **00011101**

- How about 10101001 00111001 11001100 00011001?

Constant-weight codes

- **Definition:** All non-separable codewords have the same number of 1s *(m-out of-n code)*
  - *Can detect all unidirectional errors*
  - *Maximum number of codewords obtained when the weight is n/2*

A weight-2 code:	A weight-1 code	A weight-3 code
00011	000001	000111
10100	000010	001110
00110	000100	011100
01001	001000	111000
01010	010000	001011
01100	100000	010110
10001		101100
10010		…
10100		110100
11000		111000

Arithmetic weight errors

Unsigned addition	0010 0111 0010 0001
	+ 0101 1000 1101 0011
Correct sum	0111 1111 1111 0100
Erroneous sum	1000 0000 0000 0100

How a single carry error can lead to an arbitrary number of bit-errors (inversions)

errornous carry of 1

- ***Arithmetic weight errors*:** Minimum number of signed powers of 2 that must be added to the correct value to turn it into the erroneous result (contrast with Hamming weight of an error)

Example 1	Example 2
0111 1111 1111 0100	1101 1111 1111 0100
1000 0000 0000 0100	0110 0000 0000 0100
$16 = 2^4$	$-32752 = -2^{15} + 2^4$
0000 0000 0001 0000	−1000 0000 0001 0000
1	2
Single, positive	Double, negative

Codes for arithmetic operations

- Arithmetic error-detecting codes:
  - characterized by arithmetic weights of detectable errors
  - allow direct arithmetic on coded operands
- Two classes of *arithmetic error-detecting codes*, based on a check modulus *A* (usually a small odd number)

  Product or *AN* codes

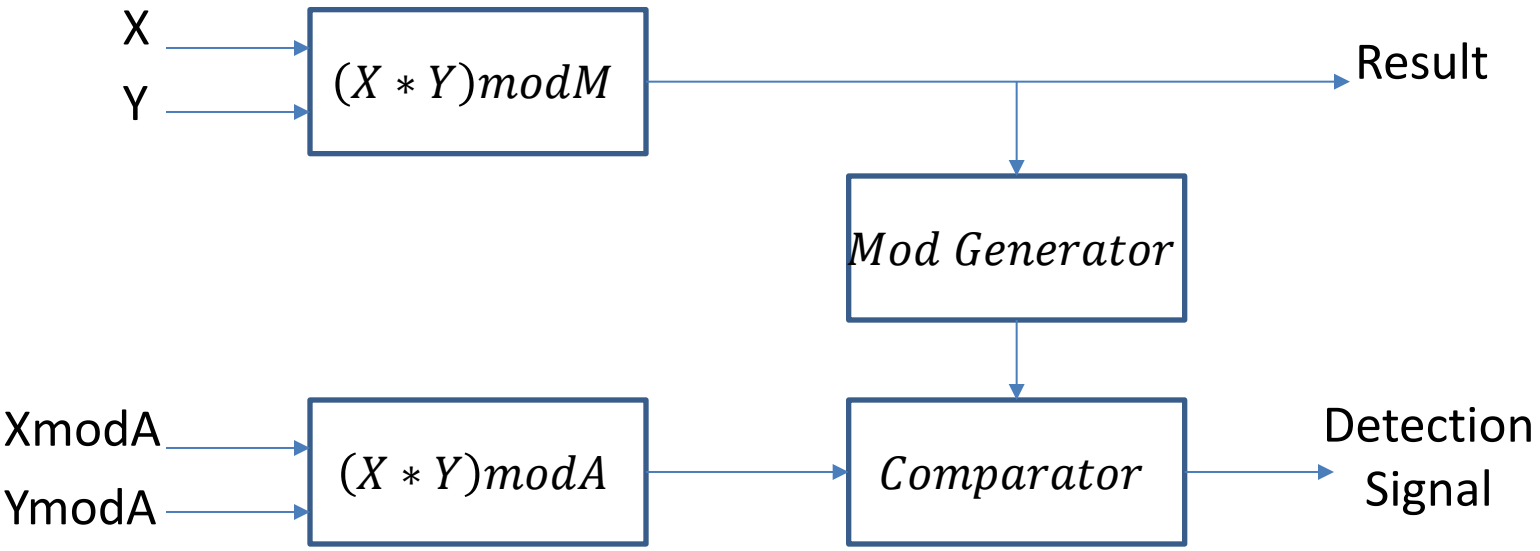
  Represent the value *N* by the number *AN*

  Residue (or inverse residue) codes

  Represent the value *N* by the pair (*N*, *C*),
  where *C* is *N* mod *A* or (*N*–*N* mod *A*) mod *A*

Residue Code

- If $MmodA = 0$, then
  $|(X * Y)modM|modA = \big((XmodA) * (YmodA)\big)modA$
- Preserve code after operation



Low-cost Residue codes

- Represent N by separable code of the pair (N, $C(N)$), where $C(N) = N \bmod A$

  Low-cost residue codes use $A = 2^a - 1$

- To compute N mod $(2^a - 1)$, add a-bit segments of N, modulo $2^a - 1$ (no division is required)

Example: Compute 0101 1101 1010 1110 mod 15

0101 + 1101 = 0011 (addition with end-around carry)

0011 + 1010 = 1101

1101 + 1110 = 1100 The final residue mod 15

Low-Cost Residue Code

- Complex Hardware of $XmodA$
- If $A = 2^b - 1$, then

$$Xmod(2^b - 1) = \left| \sum_{i=0}^{n-1} X_i \right| mod(2^b - 1)$$

- Use *b* bit End-around carry Adder

ex. $b = 3, A = 7$,and $X = 110100011$

$$Xmod7 = (419)mod7 = 110 + 100 + 011 = 110 = 6$$

Arithmetic on Residue-coded operands

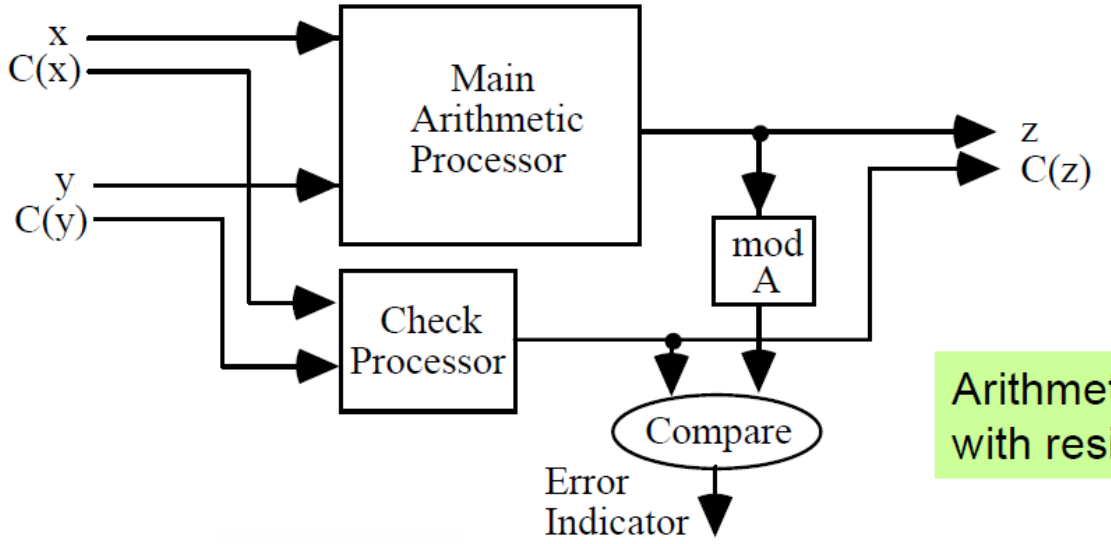
- Add/subtract: Data and check parts are handled separately

  $(x, C(x)) \pm (y, C(y)) = (x \pm y, (C(x) \pm C(y)) \bmod A)$

- Multiply:

  $(a, C(a)) \times (x, C(x)) = (a \times x, (C(a) \times C(x)) \bmod A)$

- Divide/square-root: difficult



Arithmetic processor with residue checking

Unordered Code

- A Code in which no codeword is covered by any other codeword
- Unidirectional error cannot transform one codeword to another codeword

- $X(x1, x2, \ldots, xk)$ covers $Y(y1, y2, \ldots, yk)$ is $X \geq Y$
- $X = 101010$ and $Y = 101000$ then $X \geq Y$
- If $X \geq Y$, then $h(Y, X) = 0$
- If $X \not\geq Y$ and $Y \not\geq X$, then X and Y are Unordered

m-of-n Code

- All codewords are *n* bits in length and contain exactly *m* 1s
- Non-separable and Constant Weight Code

- Simple implementation:
  - add/append second data word
  - select word such that code word contains *m* 1s
  - code is separable
  - 100% overhead
- Hamming distance is 2
  - e.g. 1st error sets bit, 2nd error resets other bit

Berger Code

- Separable Code which corresponding to the number of 1s in Data bits, and appending the bit-by-bit complement binary Check bits
- Can detect all unidirectional errors
- k Data bits wt c check bits of n bits Berger code:

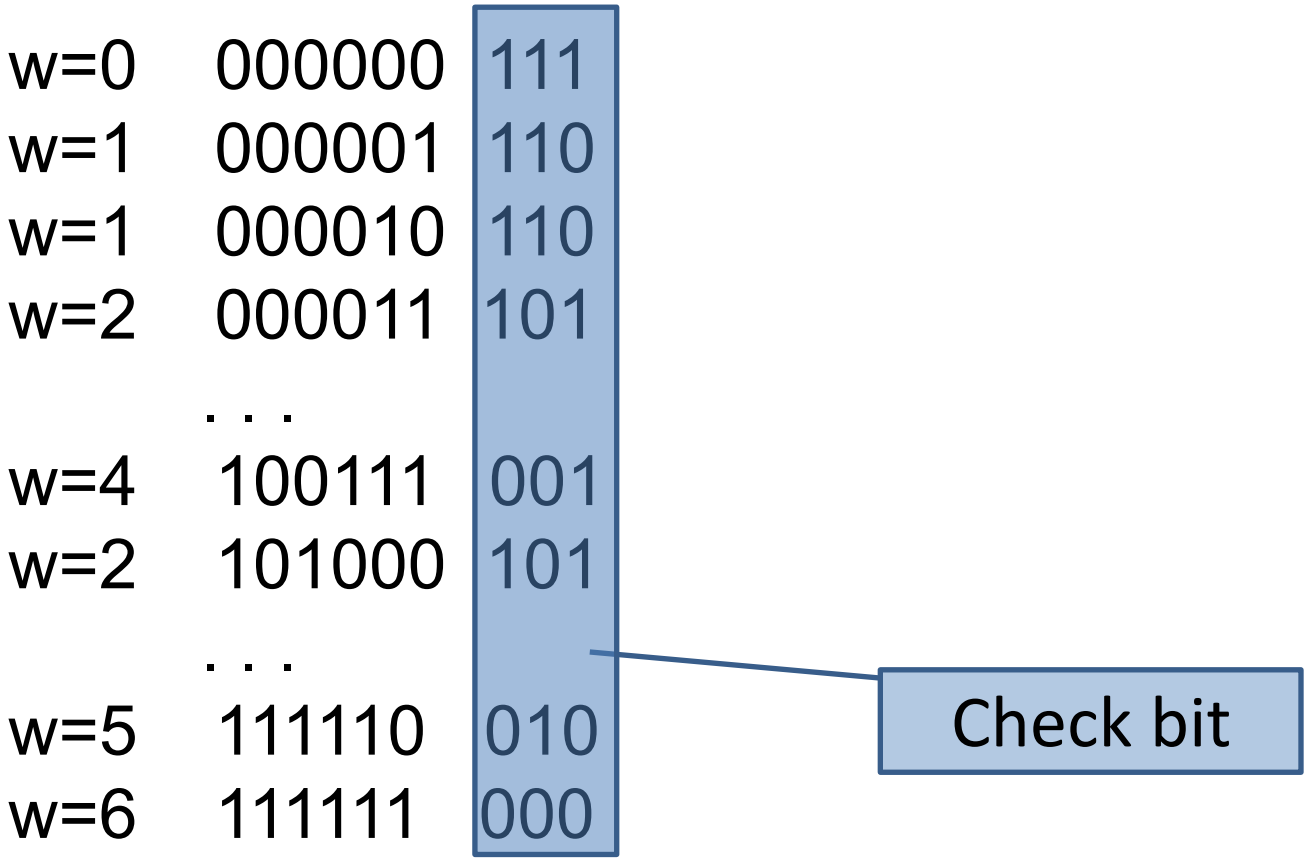
$$c = \lceil log_2(k+1) \rceil \text{ and } n = k + c$$

ex. $k = 0101000 = 2\ 1s, c = \overline{010} = 101$

$$X = 0101000101$$

Berger Code example

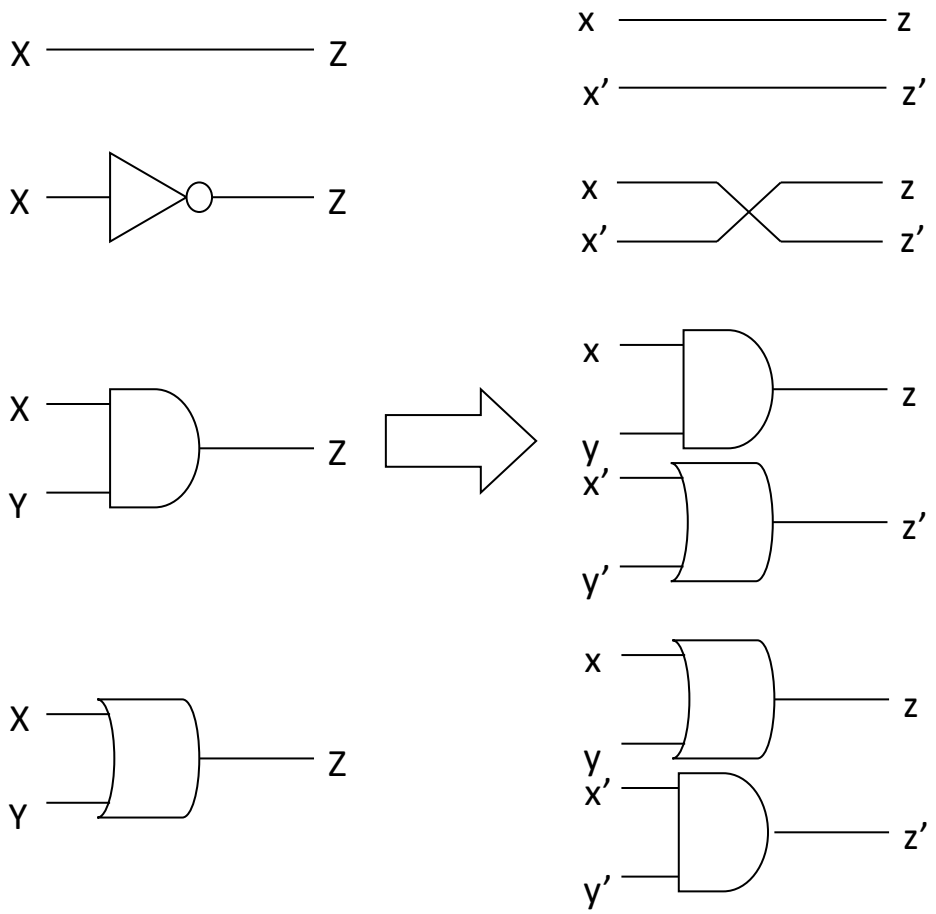
A Berger code *(k=6, c=3):*



Dual Rail Code

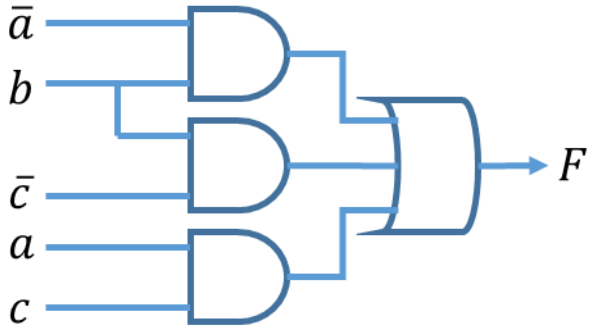
- 1-of-2 Code

  $\{0,1\} \Rightarrow \{01,10\}$
- Easy Implementation (Encoder/Decoder)
- Preserve code after operation
- Non-Inverter Circuit

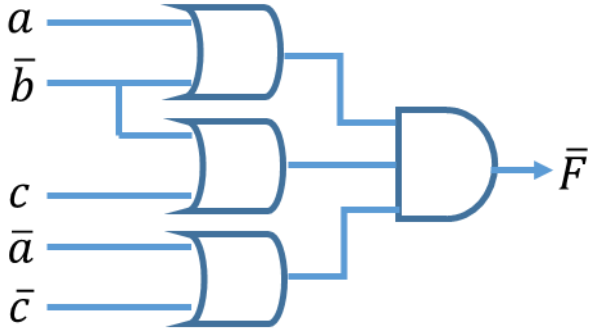


Dual Rail Code example

$$F = \bar{a}b + b\bar{c} + ac$$



$$\bar{F} = (a + \bar{b})(\bar{b} + c)(\bar{a} + \bar{c})$$



Applications: Error detection in UPC-A

• **Checksum codes:** Given a data vector $x_1, x_2, \ldots, x_n$, encode the data by attaching the checksum x_{n+1} to the end

$$(\Sigma_{j=1 \text{ to } n+1} w_j x_j = 0 \bmod A)$$

(Weight vector w_j elements are predetermined constants)

• **Capabilities:** All single-digit errors; Most transposition errors

Example: For the UPC-A checksum scheme,

• $w = 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1$; $A = 10$

• **Checking:** Verify that weighted mod-10 sum of all 12 digits is 0

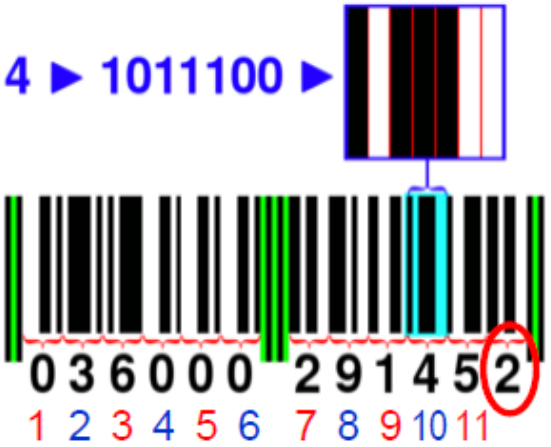
• **12-digit UPC-A universal product code:**

Sum odd indexed digits: 0+6+0+2+1+5 = 14

Multiply by 3: 14×3 = 42

Add even-indexed digits: 42+3+0+0+9+4 = 58

Compute check digit: 60 –58 = 2



Bar code uses 7 bits per digit, with different encodings on the right and left halves and different parities at various positions

Applications: JPEG Huffman Coding

- Huffman codes are widely used and very effective technique for compression data; saving of 20% to 90%
- JPEG compression standard



Original image



Reconstructed image
under Huffman code table error



Reconstructed image
wt error detection/correction